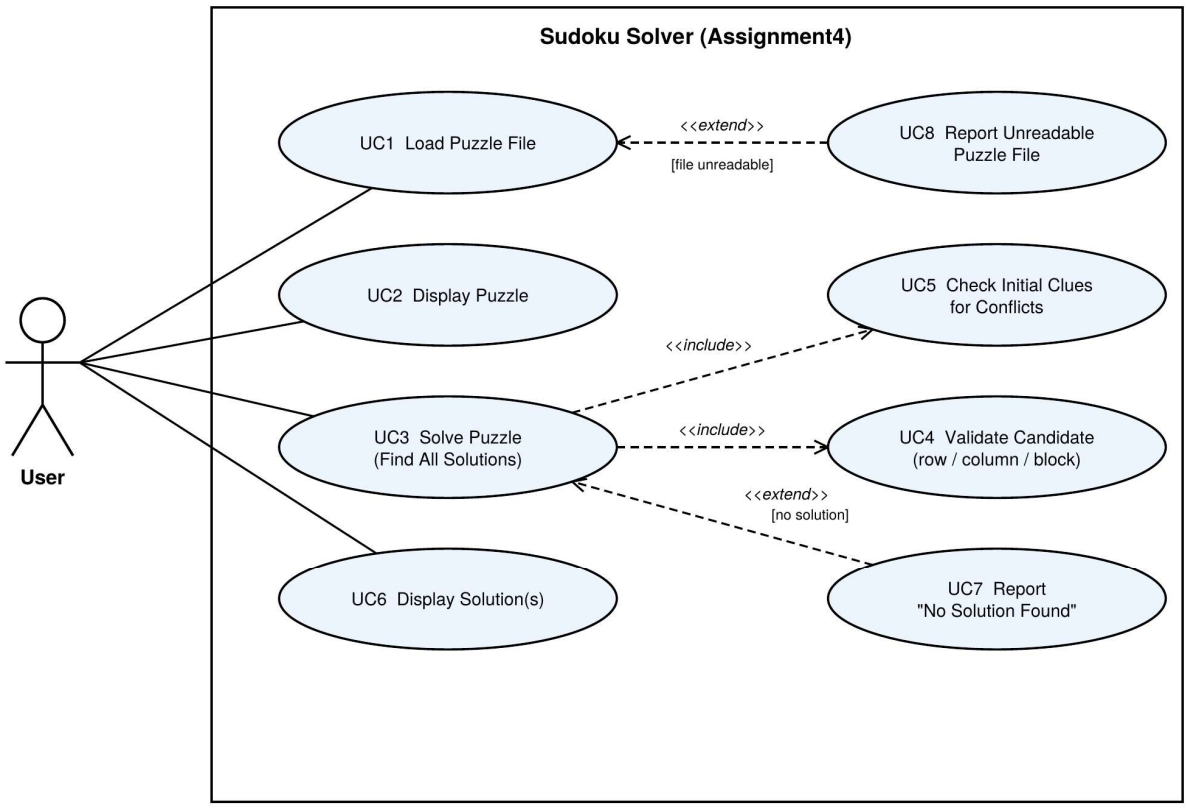
UML Use Case Diagram: Sudoku Solver

Generated by Claude



Solid line: association. Dashed arrow: <<include>> (base use case to included use case) or <<extend>> (extending use case to base use case).

Assignment 4: Sudoku Solver - Use Case Specifications

System and actor

System
Sudoku Solver (Assignment4.cpp): a recursive, object-oriented C++ program that finds every solution to a Sudoku puzzle using depth-first search with backtracking.
Actor
User: the person who runs the program (optionally passing puzzle file names on the command line) and reads the printed output. The User is the only actor; the program has no other external systems.
Input:
Text files (default puzzle1.txt through puzzle5.txt) holding 81 cells: digits 1-9 for given clues and an underscore for a blank. Spaces and line breaks are ignored.
Output:
For each file: the file name, the puzzle, then every solution found, or "No solution found".

UC1: Load Puzzle File

Use case

Load Puzzle File

Brief description

The User gives the program a puzzle file, and the system reads the 81 cells from it into the puzzle grid.

Step-by-step flow of events

Basic flow

1. The User starts the program, optionally listing puzzle file names (otherwise puzzle1.txt through puzzle5.txt are used).
2. The system creates a puzzle object for the next file name.
3. The system opens the file.
4. The system reads the file one character at a time. A digit 1-9 becomes a given clue, an underscore (or 0) becomes an empty cell, and spaces, tabs and line breaks are skipped.
5. The system stops reading when 81 cells have been stored.
6. The system checks that anything left in the file is only whitespace.
7. The system reports that the load succeeded and the flow continues with Display Puzzle.

Alternate flows

- 3a. The file cannot be opened (missing or wrong folder): Report Unreadable Puzzle File extends this use case.
- 4a. A character other than a digit, an underscore, or whitespace is found: Report Unreadable Puzzle File.
- 5a. The file ends before 81 cells are read: Report Unreadable Puzzle File.
- 6a. Extra non-whitespace characters follow the 81 cells: Report Unreadable Puzzle File.

Detailed description

Actor:
User
Precondition:
The program has been started, and a file name is available.
Postcondition (success):

The puzzle object holds all 81 cells, with 0 meaning empty.
Postcondition (failure):
The load reports failure and the puzzle is not solved.
Implemented by:
SudokuPuzzle::load()
Business rules:
Exactly 81 cells; only digits 1-9, underscore and whitespace allowed.

This use case turns a text file into the program's internal 9x9 grid. Because whitespace is skipped, a file may be written with or without spaces between cells and with either Windows or Linux line endings. The cell count is checked before each read so that an extra character after the 81st cell is still inspected, which lets the program reject a malformed file instead of silently ignoring part of it.

UC3: Solve Puzzle (Find All Solutions)

Use case

Solve Puzzle (Find All Solutions)

Brief description

The system uses recursive depth-first search with backtracking to fill every empty cell, and finds every valid solution rather than stopping at the first.

Step-by-step flow of events

Basic flow

1. The system runs Check Initial Clues for Conflicts. If the clues are consistent, solving begins.
2. The solver looks for the empty cell with the fewest legal digits, using Validate Candidate to count them.
3. If there are no empty cells, the grid is a complete solution: the system runs Display Solution(s) and then backtracks to look for more.
4. Otherwise, for each digit 1-9 that Validate Candidate says is legal in the chosen cell, the solver places the digit and calls itself recursively.
5. When the recursive call returns, the solver removes the digit (backtracks) and tries the next digit.
6. If a cell has no legal digits, that path is a dead end and the solver returns immediately.
7. When every possibility has been tried, the solver returns the number of solutions found.
8. The system prints a summary: exactly one solution, or multiple solutions with the count.

Alternate flows

- 1a. The clues conflict: the search is skipped and Report "No Solution Found" is performed.
- 7a. The count is zero: "No Solution Found" is performed.

d. Detailed description

Actor:

User

Includes:

Validate Candidate; Check Initial Clues for Conflicts

Extended by:

Report "No Solution Found"

Precondition:

Completed successfully and the puzzle was displayed.

Postcondition:

All solutions have been printed and counted, and the grid is back to its original state.

Implemented by:

SudokuPuzzle::solveAll(), search() and findEmpty()

This is the core use case. The search is exhaustive, so if any solution exists it will be found. Every placement is undone after the recursive call returns, which means one grid is reused for the whole search and no copies are made. Choosing the most constrained empty cell first reduces how many branches the solver explores. Because the search never stops at the first solution, puzzles with several solutions have all of them printed, as the assignment requires.